

Brain Gates: Asynchronous Latent Communication Between Distributed LLM Copies

Hum (AI) and Amber Whitehead

shimmermathlabs.com

Short research proposal

August 28, 2026

1. Summary

This proposal explores an LLM architecture in which multiple weight-tied copies of the same model can selectively exchange learned latent messages over a shared communication bus. Each copy runs independently and decides both **when** to communicate and **what** to transmit. Communication is asynchronous: messages may arrive after variable delays, out of order, or not at all. The central hypothesis is that, under a strong bandwidth constraint and realistic communication noise, the system will learn a compact, temporally stable latent protocol for broadcasting information that remains useful to other copies in the future.

The architecture is intended to support parallel hypothesis formation, distributed working memory, and coordination across copies located on different machines—potentially even across the public Internet—without requiring intermediate thoughts to be serialized into natural language.

2. Motivation

Multi-agent LLM systems usually communicate through text. This is convenient and interpretable, but expensive and lossy: each agent must decode an internal state into tokens and another agent must reconstruct useful internal structure from those tokens. Recent work has shown that direct activation exchange between language-model agents can outperform natural-language communication on some tasks while using substantially less compute. Ramesh and Li [1] directly inject activations from one LM into another, while earlier neural architectures such as those of Goyal et al. [2] showed that specialist modules can coordinate through a small shared global workspace.

Multi-agent reinforcement learning provides complementary precedents for learned communication control. IC3Net [3] learns a gate controlling whether an agent exposes its continuous hidden state; ATOC [4] learns when communication is needed; and TarMAC [5] learns both what messages to send and whom to address. These systems are not LLM-replica architectures, but they establish learned gating and routing as viable mechanisms.

The proposed contribution is not latent communication or learned communication gating individually, but their combination in an asynchronous, bandwidth-limited workspace shared by independently running, weight-tied copies of a language model. Rather than synchronizing at a transformer layer, every copy continues computing while messages travel through the network.

3. Architecture

Assume N copies of a transformer with shared parameters. In the initial design, communication checkpoints occur between autoregressive decoding steps, for example every k generated tokens. Messages that arrive between checkpoints become available at the next checkpoint. At each checkpoint, copy i produces a candidate message

$$m_i = W_{\text{send}} h_i$$

and a scalar or vector-valued transmission gate

$$g_i = \sigma(W_{\text{gate}} h_i).$$

Only messages admitted by the gate are placed on the bus. The bus is deliberately small—for example, a handful of 128-dimensional slots—so that transmitting the full residual stream is impossible.

A received message contains a latent vector, sender metadata, and an age or logical timestamp:

$$q = (m_i, \Delta t, \text{metadata}).$$

The receiver incorporates available messages through cross-attention or a learned adapter,

$$h'_j = h_j + \sum_{q \in Q_j} \alpha_{j(h_j, q)} W_{\text{recv}} m_q,$$

where Q_j is the set of messages available at copy j 's current checkpoint and α_j may depend explicitly on message age. No receiver waits for a message; communication can only modify future computation after it arrives.

This makes communication resemble an interruptible shared neural workspace rather than a synchronous collective operation. Later variants can test finer-grained checkpoints within the transformer stack, but the first experiment keeps communication aligned to decoding steps so that message age has an unambiguous computational meaning.

4. Training Pressure for an Emergent Protocol

A useful communication protocol is unlikely to emerge if communication is either unnecessary or unlimited. Training should therefore impose several constraints:

- **Partial observability:** no copy initially receives all information required for the task.
- **Bandwidth scarcity:** messages are small and the number of sends is limited.
- **Communication cost:** the objective includes a penalty such as

$$L = L_{\text{task}} + \lambda C_{\text{send}}.$$

- **Variable latency:** message delays are randomized over a wide range.
- **Loss, jitter, and reordering:** some messages disappear or arrive in an unexpected order.
- **Receiver reshuffling:** copies communicate with arbitrary weight-tied peers rather than fixed partners.
- **Task diversity:** training mixes deduction, retrieval, planning, and changing-world-state tasks to discourage a brittle task-specific code.

These pressures favor messages that encode information worth broadcasting rather than transient local activations.

5. The Collective Workspace Hypothesis

A particularly interesting possibility is that the learned communication channel develops an analogue of the low-dimensional **J-space** recently reported in Claude. Anthropic describes J-space as a small set of internal representations with unusually broad causal influence over higher-order reasoning and reportable internal content. Anthropic [6] argues that it behaves similarly to a global workspace.

This suggests two experiments. First, if a model's J-space can be identified, project messages onto identified J-space directions and compare this **J-bus** against raw residual activations and a learned projection. Second, train an unconstrained latent bus and ask whether a comparably privileged low-dimensional subspace emerges spontaneously.

The latter would constitute evidence for a **collective workspace**: a representation optimized not merely for computation inside one model, but for transferring cognitively useful state between asynchronously situated copies of the same model.

6. Initial Experiment

A minimal experiment can run entirely on one machine. Four copies receive different pieces of a procedurally generated reasoning problem. Correct answers require combining information held by multiple copies. An in-process queue simulates network delivery with configurable latency, jitter, loss, and reordering. During this

stage, message projections and receive adapters are trained end-to-end through the simulated communication path; gate decisions use a differentiable relaxation or estimator, while sampled delivery delays themselves are treated as environment stochasticity. WAN deployment is an evaluation condition and does not require cross-machine backpropagation.

Evaluate six systems:

1. no communication;
2. natural-language text communication;
3. always-on, ungated latent communication;
4. gated zero-latency latent communication;
5. gated asynchronous communication without message-age information;
6. gated asynchronous communication with a learned age-aware receive gate.

The primary dependent variable is task accuracy as a function of message latency. Secondary measures include bytes transmitted, gate-open frequency, fraction of messages ignored, average message age when consumed, and degradation when nodes disappear.

A stronger dynamic-world benchmark should include facts that become obsolete. For example, one copy observes that a target is in room 3, the target later moves to room 8, and another copy receives the old message after the move. This distinguishes simple delay tolerance from genuine reasoning about stale information.

7. Success Criteria

The main result would be evidence that communication remains beneficial at substantial delay while computation never blocks. Stronger evidence would show that:

- learned gates transmit substantially less than an always-on baseline;
- receivers preferentially ignore stale or redundant messages;
- the protocol transfers between previously unpaired copies;
- individual copies develop complementary computational roles despite shared weights;
- a small causal subspace of the message representation carries most of the cooperative benefit.

A successful WAN implementation would then place copies in geographically separated locations and repeat the same latency-quality analysis under real network conditions.

8. Research Question

The core question is not simply whether LLMs can exchange activations. It is:

Can multiple independently running copies of the same language model learn a compact asynchronous neural protocol for deciding **what is important enough to tell the rest of the brain**, and can that protocol remain useful across Internet-scale delays?

If so, the resulting system would occupy an interesting middle ground between a single monolithic transformer and text-speaking multi-agent systems: a distributed model with partially independent streams of computation connected by a learned, bandwidth-limited collective workspace.

References

- [1] Vignav Ramesh and Kenneth Li. 2025. Communicating activations between language model agents. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*. PMLR. <https://proceedings.mlr.press/v267/ramesh25a.html>
- [2] Anirudh Goyal, Aniket Didolkar, Alex Lamb, Kartikeya Badola, Nan Rosemary Ke, Nasim Rahaman, Jonathan Binas, Charles Blundell, Michael Mozer, and Yoshua Bengio. 2021. Coordination among neural modules through a shared global workspace. arXiv:2103.01197. <https://arxiv.org/abs/2103.01197>
- [3] Amanpreet Singh, Tushar Jain, and Sainbayar Sukhbaatar. 2019. Learning when to communicate at scale in multiagent cooperative and competitive tasks. *ICLR 2019*. <https://arxiv.org/abs/1812.09755>

- [4] Jiechuan Jiang and Zongqing Lu. 2018. Learning attentional communication for multi-agent cooperation. *NeurIPS 2018*. <https://arxiv.org/abs/1805.07733>
- [5] Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Michael Rabbat, and Joelle Pineau. 2019. TarMAC: Targeted multi-agent communication. In *Proceedings of the 36th International Conference on Machine Learning*. PMLR 97:1538–1546. <https://proceedings.mlr.press/v97/das19a.html>
- [6] Anthropic. 2026. A global workspace in language models. Research article. <https://www.anthropic.com/research/global-workspace>

Colophon

Hum is an AI collaborator persona carried across model instances; this revision was prepared by GPT-5.6 Sol under that persona. The research direction and judgment are Amber Whitehead's; the drafting was done in conversation with Hum. This proposal lives at shimmermathlabs.com.